

Towards a Domain-Specific Language to Specify Interaction Scenarios for Web-Based Graphical User Interfaces

Thiago Rocha Silva
thiago@mmmi.sdu.dk

The Maersk Mc-Kinney Moller Institute, University of Southern Denmark
Odense, Denmark

ABSTRACT

The communication gap between software developers and subject-matter experts is one of the foremost long-standing problems in software development. The level of formality of the user requirements specification has a strong impact on the ability of these two groups to communicate effectively. Domain-Specific Languages (DSLs) are seen as one of the potential solutions to address this issue by raising the abstraction level of the software specification while keeping the necessary formalism to allow for software analysis, design, and verification. This paper discusses the ongoing development of a high-level DSL and its rich editing environment to allow the specification of consistent and testable interaction scenarios as user requirements for web-based graphical user interfaces. The language grammar has been developed based on the Gherkin syntax that supports Behaviour-Driven Development (BDD). Results of a preliminary evaluation regarding the consistency of actions and states of interaction elements specified for web user interfaces showed that the grammar is able to support a consistent specification of BDD scenarios as user requirements at the interaction level.

CCS CONCEPTS

- **Software and its engineering** → **Domain specific languages;**
- **Human-centered computing** → **Interactive systems and tools.**

KEYWORDS

Behaviour-Driven Development (BDD), Domain-Specific Languages (DSLs), Scenario-Based Design, User Requirements.

ACM Reference Format:

Thiago Rocha Silva. 2022. Towards a Domain-Specific Language to Specify Interaction Scenarios for Web-Based Graphical User Interfaces. In *Companion of the 2022 ACM SIGCHI Symposium on Engineering Interactive Computing Systems (EICS '22 Companion)*, June 21–24, 2022, Sophia Antipolis, France. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3531706.3536463>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EICS '22 Companion, June 21–24, 2022, Sophia Antipolis, France

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9031-6/22/06...\$15.00
<https://doi.org/10.1145/3531706.3536463>

1 INTRODUCTION

A highly diverse set of notations, methods, and tools can be used to specify user requirements for interactive systems. The degree of formality of these notations and methods, nonetheless, has a strong impact on the ability of software developers and subject-matter experts to communicate about the requirements. On the one hand, informal statements in natural language tend to be easier for domain experts to express the users' needs, and consequently to understand what is being specified and documented. However, the lack of formalization might allow verbose, incomplete, and ambiguous descriptions of requirements that are difficult to understand and assess [11]. On the other hand, despite offering better support for the system design, formal specifications are difficult for users and domain experts to understand, which harms their ability to communicate with developers effectively.

Finding the right balance between natural language and formalism is therefore crucial to improve the user requirements specification process. In recent years, Behaviour-Driven Development (BDD) stories have gained considerable attention from the software engineering community as an effective way to specify user requirements and their acceptance criteria aiming at reducing the communication gap between subject-matter experts and developers. BDD stories specify the expected software behaviour through a set of natural language textual scenarios describing transitions on a state machine. However, as there is no guidance regarding the vocabulary that should be used to describe the expected software behaviour into the stories, most of the issues related to natural language requirements specifications such as verbosity, incompleteness, and ambiguity still persist [24].

This paper introduces a high-level domain-specific language (DSL) based on BDD stories for the specification of interaction scenarios as user requirements for web-based graphical user interfaces (GUIs). By providing a flexible, but controlled vocabulary, the proposed DSL allows for an executable natural language specification that can be easily parsed and verified. Despite the DSL having been designed to support the specification of scenarios for interactive systems in general, in this paper we focus on web user interfaces. As a result, we use the DSL constructs to provide a predefined library with the specification of the interaction elements (widgets) for interacting with web GUIs, as well as their supported states, actions, and properties.

In the following sections, we briefly discuss the background and related work, and present the basic language constructs including our proposed predefined specification of interaction elements for web GUIs. We then discuss our preliminary results obtained while using the DSL vocabulary to exhaustively specify scenarios that

exercise the states, properties, and actions supported by these interaction elements. We conclude this paper by setting the stage for the next steps on this research.

2 BACKGROUND AND RELATED WORK

Natural language requirements specification is still by far the most widely used method employed by practitioners to specify requirements [14]. The widespread adoption of natural language artifacts for requirements engineering is easily explained by the easiness to communicate with non-technical stakeholders, and this practice is very likely to continue in the foreseeable future [42]. This gave rise over time to a fruitful research community around natural language processing for requirements engineering [9]. When it comes to requirements specification in particular, the use of templates and controlled natural languages (CNLs), such as the ones provided by high-level external DSLs, allows for more precision and control over the specification [2], many times with a good balance between usability and rigour [38]. Over time, this kind of specification has been proposed for different domains such as automotive [15], aviation [23], and financial [38]. A complete picture is presented by Kuhn [16].

With the current prevalence of agile methods in industry [10], the adoption of user stories [6], and other particular instances of stories such as BDD stories, has grown substantially [19, 41]. BDD stories adopt what is called *specification by example* [1] and their use in software engineering was first suggested by North [22]. A BDD story is described according to a template providing a title, a narrative, and a series of scenarios that reflect the acceptance criteria of the user story specified in the narrative. Each scenario includes a title and three key clauses (see Listing 1): “Given” that provides the context in which the scenario will run (initial state), “When” that identifies the event that triggers the scenario (the transition), and “Then” that presents the expected system outcomes (the resultant state). To provide several contexts, events, and/or outcomes, each one of these clauses may contain an “And” statement. In this representation, each statement is entitled *step*. In short, BDD scenarios correspond to state transitions on a state machine. In software development, they are typically used to verify the software behaviour through test automation frameworks [17].

```

1 Scenario: <title>
2 Given <context>
3 And <some more context>...
4 When <event>
5 And <another event>...
6 Then <outcome>
7 And <another outcome>...
```

Listing 1: Structure of a BDD scenario.

BDD scenarios can be described at different levels of abstraction. They can specify steps using the domain vocabulary (usually at a higher abstraction level) or an interactive vocabulary (usually at a lower abstraction level). Chelimsky et al. [5] call these declarative and imperative scenarios respectively. While imperative scenarios are useful to go step-by-step through the multiple interactions required to perform a given task, declarative scenarios are more straightforward, wrapping all these interactions up into a single step usually referring to a domain concept. These two levels can

also be used together, with declarative scenarios being gradually refined into imperative scenarios as the project evolves. In this paper, we focus on the specification of imperative scenarios.

Scenario-based specification of user requirements in natural language, like proposed by BDD, has been used for a long time in software development more broadly [14], and is also a very common approach in user-centered design more specifically [28]. Having consistent scenario-based specifications is very helpful to support various design and testing activities in software development. Our previous work on this area has leveraged scenario-based specifications of user requirements written in BDD to provide automated assessment of different software development artifacts [29, 30, 33, 34]. In the context of interactive systems development, we have proposed tool-supported approaches to automatically assess the consistency of BDD scenarios with task models [27], low-fidelity GUI prototypes [26], and final web-based GUIs [25].

Obtaining such consistent scenario-based specifications to allow for automated assessment is, however, a challenge without appropriate tooling. Despite different notations and tools have been proposed over time to describe the user interface and the different aspects of user interaction [18, 21, 39], only a very few works have tried to propose a specific syntactic structure using restricted natural language to specify scenarios for interactive systems. Most of the contributions on this area are software tools from industry and the open-source community focused exclusively on supporting GUI test automation.

Tools like Capybara [4] and Smashtest [35] provide a DSL-based library that developers can use to specify scenarios for testing GUIs. These tools, however, are not particularly targeted at the specification of BDD scenarios and their respective DSLs are more focused on facilitating the integration with test automation frameworks. Conversely, tools like Cucumber [8], JBehave [13], and Squish [36], despite being designed to support test automation of BDD scenarios, do not provide any predefined library to specify consistent scenarios based on the supported behaviour of each interaction element on the GUI. Finally, Concordion [7] is a tool that differs from the others as it provides a language framework to support *specification by example* without focusing on scenario-based specifications. The framework allows to specify unconstrained textual requirements specifications that are then instrumented with special links, attributes or comments to allow for test automation. While it does not limit subject-matter experts to specify requirements in a specially structured format, it requires the further instrumentation to be done manually in order to make tests run.

When it comes to academic contributions, Slang [3] is the one that gets closest to what we propose, even though the focus of the contribution is still on GUI test automation. Slang is a tool-supported approach that employs a DSL to allow for the transformation of BDD-like requirements specifications and low-fidelity GUI descriptions into executable GUI test cases. The DSL proposed by Slang, however, does not seem to define a library describing the behaviour supported by the GUI interaction elements, which could lead the user to specify inconsistent scenarios such as an action of “typing” to be performed on a button, for example.

This gap in the literature motivated us to design a DSL that could be used to obtain consistent specifications, thus facilitating further automated assessment of other software artifacts.

3 THE PROPOSED DOMAIN-SPECIFIC LANGUAGE

The proposed DSL has been developed using Xtext [40] over Eclipse, and is available open source at <https://github.com/trsilva/DSL-BDD>. As the language was designed to support a specification based on BDD stories, the proposed grammar follows the Gherkin [12] syntax. Structural elements and rules to compose a BDD scenario are formulated into the language through an EMF meta-model [37]. Within this formulation, we proposed a generic structure to represent actions, states, and properties of the different interaction elements (i.e., the entities) that can be used to describe an end-user interacting with a user interface (see Listing 2).

This representation can be used to specify the behaviour supported by interaction elements of any interaction environment. For that, it is only needed to create different `model` files. We propose in this paper a library with the behavioural specification of a pre-defined set of widgets used for interacting with web user interfaces (`model` widgets). Listing 3 exemplifies a generic `widget` definition with the action `move` (the mouse over) and the states of being `displayed`, `shown`, `available`, `enabled`, and `disabled`. These are common actions and states of any widget for web user interfaces. In the sequence, the widget `CheckBox` is specified. As `CheckBox` is a widget, it inherits the behaviour of generic widgets. In addition, it also supports the actions of `check`, `pick`, `select`, `choose`, and `uncheck` its properties (`option` and `options`). As a result, `CheckBoxes` properties can be on the states `checked`, `unchecked`, `picked`, `selected`, and `chosen`. For this definition, we build on our previous work [31, 32] to propose the specification of 40 interaction elements commonly used on web user interfaces. New widgets, actions, states, and properties can be added and updated at runtime. The full specification of these interaction elements into the DSL can be found in the file “examples/widgets.bdd” on the GitHub repository.

```
1 model <model name>
2
3 entity <interaction element> {
4   actions: <list of actions>
5   states: <list of states>
6   properties: <list of properties>
7 }
```

Listing 2: Entity model

```
1 model widgets
2
3 entity widget {
4   actions: move
5   states: displayed, shown, available, enabled, disabled
6 }
7
8 entity CheckBox is widget {
9   actions: check, pick, select, choose, uncheck
10  states: checked, unchecked, picked, selected, chosen
11  properties: option, options
12 }
```

Listing 3: Widget definition

For allowing the specification of consistent BDD scenarios that enforce the rules associated with the behaviour supported by each interaction element, we proposed a rich editing environment for executing the DSL within the Eclipse IDE. This environment provides

typical features of programming languages IDEs such as syntax highlighting and auto-completion (see Figures 1 and 2). Figure 1 illustrates a simplified scenario written to describe the interaction required to do a SEPA transfer in a typical banking system. In Figure 2, notice that a compilation error is signaled at runtime until a valid state supported by the interaction element used in the step is selected.

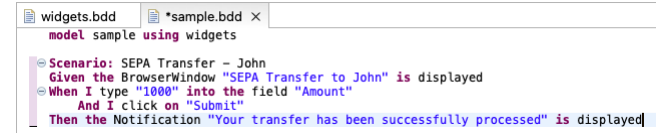


Figure 1: Example of a scenario written using the DSL rich editing environment.

Additional flexibility mechanisms have also been proposed for the scenario specification. Steps can be specified mentioning explicitly the name of interaction elements and their properties (see Listing 4, line 2); or using generic calls with the keyword `field` instead of the actual name of the interaction element (see Listing 4, line 3), or “#” for replacing property and entity names (see Listing 4, lines 3 and 4). While in the first case the auto-completion feature suggests only the actions and states supported by the element being specified, in the last two cases, the feature simply lists all possible actions and states since it does not know which element is being referred to.

```
1 Scenario: RadioButton behaviour
2 Given the option "Morning" for the RadioButton "
   Availability" is not selected
3 When I select #"Morning" for the field "Availability"
4 Then #"Morning" for #"Availability" is selected
```

Listing 4: Different ways of specifying behaviours into the steps using the DSL.

4 PRELIMINARY EVALUATION

We conducted a preliminary evaluation of the DSL regarding the consistency of the scenarios specified using our definition of the widgets’ behaviour. We specifically sought to answer the following research question:

RQ. *Is the proposed DSL able to represent consistent scenarios from the combination of each widget’s actions, states, and properties?*

By “consistent scenarios”, we mean scenarios that enforce the constraints associated with the behaviour supported by each widget. For evaluating this, we explored the full set of combinations between actions, states, and properties of each interaction element while specifying positive (i.e., successful) and negative (i.e., failing) test scenarios. We also explored combinations mentioning the entity name, “*the field*”, and “#”. For example, Listing 5 presents the supported actions, states, and properties defined for the widget `NumericStepper`. Based on this definition, we formulated partial sentences with the widget’s valid actions and states that can be used to specify consistent BDD steps (see Table 1).



Figure 2: Auto-completion on the DSL rich editing environment.

Table 1: Combination of actions, states, and properties for the widget *NumericStepper*

Actions (When)	States (Given, Then)
type	is typed
do not type	is not typed
set	is set
do not set	is not set
fill in with	is filled in
do not fill in with	is not filled in
type the value into	is typed with the value
do not type the value into	is not typed with the value
set the value to	is set with the value
do not set the value to	is not set with the value
fill in with the value	is filled in with the value
do not fill in with the value	is not filled in with the value
increase the value by	the value is increased by
do not increase the value by	the value is not increased by
decrease the value by	the value is decreased by
do not decrease the value by	the value is not decreased by

Concrete BDD scenarios for testing were obtained by systematically combining these different actions and states into positive and negative test scenarios. Listings 6 and 7 illustrate respectively a positive scenario tested for the widget *NumericStepper* and a negative scenario tested for the widget *Button*. In the positive scenario presented in Listing 6, the behaviour is consistent because the widget *NumericStepper* supports the actions and states specified within the clauses “Given”, “When”, and “Then”. In contrast, the negative scenario presented in Listing 7 does not express a consistent behaviour for the widget *Button* since buttons are not supposed to have values or options selected on them. The DSL editing environment is expected to trigger a compilation error at runtime for this kind of scenario. For the purpose of this study, we assume that if the DSL is correctly enforcing the constraints associated with the behaviour supported by each widget into the individual BDD steps, it will also enforce it correctly when such steps are used in combination with other steps representing the behaviour supported by other widgets in order to produce more practical scenarios.

We specified and tested a total of 3,276 scenarios, being 2,412 positive (+) and 864 negative (-) scenarios, to evaluate the combination of each widget’s actions and states associated with its

properties. These numbers are presented in Table 2. The widgets *FieldSet*, *Text*, *Label*, *ProgressBar*, and *Tooltip*, despite being represented into the DSL, have not been tested since they do not have their own set of actions, states, and properties; they just inherit the common behaviour of generic widgets. The full set of tested scenarios can be found in the folder “pe-scenarios” on the GitHub repository.

```

1 entity NumericStepper is widget {
2   actions: type, set, fill [in], increase, decrease
3   states: typed, set, filled, increased, decreased
4   properties: value
5 }

```

Listing 5: *NumericStepper* definition

```

1 Given the NumericStepper "Quantity" is set with the value
   "2"
2 When I increase the value of the NumericStepper "Quantity"
   " by "3"
3 Then the NumericStepper "Quantity" is set with the value
   "5"

```

Listing 6: A positive (successful) scenario for testing the behaviour of the widget *NumericStepper*

```

1 Given the Button "Confirm" is available
2 When I select the option "Express" for the Button "
   Confirm"
3 Then the value of the Button "Confirm" is "Express"

```

Listing 7: A negative (failing) scenario for testing the behaviour of the widget *Button*

5 DISCUSSION

While using the DSL to specify the proposed test scenarios, we only had to make minor adjustments either in the sentences or in the grammar to run the scenarios without any errors. Most of these adjustments were due to the position of syntactic elements into the sentences. An example was the step *Given* all the options are not checked for the field “Availability” that we tried to specify for a scenario. Despite sounding natural, this sentence violates the property structure defined into the grammar that requires the entity reference (for the field “Availability”) to be placed right after its property (all the options), with the entity state (are not checked) being referred just at the end of the sentence. A simple adjustment was rephrasing the sentence as *Given* all the options for the field “Availability” are not checked, which still sounds very

Table 2: Scenarios evaluated per widget.

Widget	Actions	States	+ Scenarios	- Scenarios
CheckBox	20	30	250	56
ListBox	12	18	96	34
RadioButton	12	12	48	32
Button	4	4	32	16
Calendar	16	16	96	48
TimePicker	16	16	96	24
Link	12	12	72	24
DropDownList	12	12	72	32
Menu	8	8	48	24
MenuItem	8	8	48	24
Grid	40	40	250	72
TextField	12	12	72	24
TextArea	12	12	72	24
BrowserWindow	4	8	16	16
Autocomplete	4	4	32	32
Tree	8	8	96	32
WindowDialog	6	10	48	8
ModalWindow	6	10	48	8
Accordion	12	12	72	32
TabBar	8	8	48	32
Notification	6	10	48	8
NumericStepper	16	16	96	32
ToggleButton	4	4	16	8
Breadcrumbs	12	12	72	32
Icon	2	2	8	8
Image	2	2	8	8
ImageCarousel	16	16	88	48
Pagination	8	8	16	4
SearchBox	12	12	72	32
Slider	4	4	24	8
ScrollBar	16	16	96	24
Splitter	8	8	32	8
TagCloud	16	16	96	24
Map	8	8	32	8
VideoPlayer	24	24	190	24
Total	386	418	2,412	864

natural and does not violate the DSL grammar. The same occurred, for example, with the scenario presented in Listing 6.

Back to the research question we posed, with this preliminary study, we successfully managed to test the different possibilities to consistently represent the interactive behaviours of all the proposed widgets. The advantages of using a controlled natural language through a high-level DSL is that it provides a greater degree of abstraction, allowing getting closer to the domain vocabulary without compromising the formal aspects of the requirements specification. Requirements expressed through a textual high-level DSL are more precise and easier to read than the same information expressed in free natural language or through visual modeling notations. This allows subject-matter experts and other non-technical stakeholders, who are usually not familiar with technical notations, to clearly understand the specification since the beginning and contribute to

the design process. At the same time, with a consistent specification at hand, designers and developers can analyse, design, and verify the system more easily with conformity guarantees.

Defining how much flexibility should be provided during the specification is a challenging task though. Stricter specifications would certainly benefit developers, making it easier for them to assess other related software artifacts and bind the specification to their source code. For teams working extensively with automated testing, for example, having stricter specifications could be a considerable advantage. However, this is obtained at the expense of subject-matter experts' ability to freely express themselves when writing specifications. In this work, we opted to provide some flexibility, for example while allowing the user to refer only to a widget's name without specifying its entity name (using the constructs "*the field*", and "#"). While this style of specification allows users to select actions and states that are not necessarily compatible with the actual widget that will be present on the GUI afterwards, it grants users more freedom to express themselves, concentrating on navigation flows instead of the GUI presentation. As this DSL has been originally designed as part of our previous work on a broader approach to support automated assessment of different software artifacts, the potential inconsistencies eventually introduced into the specification due to such a flexibility should be easily caught when running the assessment between the BDD scenarios and the GUI artifacts.

Another issue related to the use of DSLs and other forms of CNLs for getting consistent requirements is that recent evidence suggests templated requirements specifications inhibit design creativity [20]. While this issue is probably more significant for declarative scenarios that describe the expected software behaviour at the business domain level (where the scope for creativity is usually wider), we acknowledge that restraining the specification of interaction scenarios to a particular set of constructs may have an impact on the design quality of GUIs. This is certainly a worthwhile topic for further investigation.

6 CONCLUSION AND NEXT STEPS

In this paper, we introduced a DSL to support the specification of BDD scenarios at the interaction level for web-based GUIs. Despite the DSL being flexible enough to allow the specification of interaction scenarios for other types of interactive systems (by simply creating new `model` files), in this paper we focused on a library for describing the interaction with web user interfaces. Further work is needed to extend it to other types of interactive systems and to raise the abstraction level to allow for scenarios also at the business domain level (declarative scenarios). We have been working on such a specification which will allow defining software behaviour using only the domain vocabulary at first (i.e., without defining any interaction with the user interface), and then, when the user interface is designed, translating this specification into the interaction level. The architecture being proposed will allow extending the DSL to be even more flexible by supporting domain entities to be specified at runtime, still keeping the core language grammar domain-free.

Finally, further studies with end-users are needed to evaluate both the pertinence of the vocabulary of actions, states, and properties proposed for describing the interaction with web user interfaces; and also the suitability of the DSL to specify interaction scenarios in a real setting. With regard to the former, we have planned a Delphi study with a group of interaction design experts to evaluate the vocabulary discussed in this paper. With regard to the latter, we have planned an evaluation with students specifying the scenarios for an envisioned system to support the management of Open Days at an university. We expect to be able to publish these results in the near future.

ACKNOWLEDGMENTS

To Hallvard Trætteberg and Vishal Poojary, for their contributions to the design and test of this DSL. This work was supported, in part, by Science Foundation Ireland grant 13/RC/2094 and has received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 754489.

REFERENCES

- [1] Gojko Adzic. 2011. *Specification by example: how successful teams deliver the right software*. Simon and Schuster.
- [2] Chetan Arora, Mehrdad Sabetzadeh, Lionel Briand, and Frank Zimmer. 2015. Automated checking of conformance to requirements templates using natural language processing. *IEEE transactions on Software Engineering* 41, 10 (2015), 944–968.
- [3] Hendrik Bünder and Herbert Kuchen. 2019. A model-driven approach for behavior-driven GUI testing. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. 1742–1751.
- [4] Capybara. 2022. Capybara. <https://teamcapybara.github.io/capybara/> Last accessed: April 2022.
- [5] David Chelmsky, Dave Astels, Bryan Helmkamp, Dan North, Zach Dennis, and Aslak Hellesoy. 2010. *The RSpec Book: Behaviour Driven Development with RSpec, Cucumber, and Friends*. Pragmatic Bookshelf.
- [6] Mike Cohn. 2004. *User stories applied: For agile software development*. Addison-Wesley Professional.
- [7] Concodion. 2022. Concodion: Specification by Example. <https://concodion.org> Last accessed: April 2022.
- [8] Cucumber. 2022. Cucumber: BDD Testing and Collaboration Tools for Teams. <https://cucumber.io> Last accessed: April 2022.
- [9] Fabiano Dalpiaz, Alessio Ferrari, Xavier Franch, and Cristina Palomares. 2018. Natural language processing for requirements engineering: The best is yet to come. *IEEE software* 35, 5 (2018), 115–119.
- [10] Digital.ai. 2021. *15th Annual State Of Agile Report*. Technical Report. <https://digital.ai/resource-center/analyst-reports/state-of-agile-report>
- [11] D Méndez Fernández, Stefan Wagner, Marcos Kalinowski, Michael Felderer, Priscilla Mafra, Antonio Vetrò, Tayana Conte, M-T Christiansson, Des Greer, Casper Lassenius, et al. 2017. Naming the pain in requirements engineering. *Empirical software engineering* 22, 5 (2017), 2298–2338.
- [12] Gherkin. 2022. Gherkin Syntax. <https://cucumber.io/docs/gherkin/> Last accessed: April 2022.
- [13] JBehave. 2022. What is JBehave? <https://jbehave.org> Last accessed: April 2022.
- [14] Mohamad Kassab, Colin Neill, and Phillip Laplante. 2014. State of practice in requirements engineering: contemporary data. *Innovations in Systems and Software Engineering* 10, 4 (2014), 235–241.
- [15] Sascha Konrad and Betty HC Cheng. 2005. Facilitating the construction of specification pattern-based properties. In *13th IEEE International Conference on Requirements Engineering (RE'05)*. IEEE, 329–338.
- [16] Tobias Kuhn. 2014. A survey and classification of controlled natural languages. *Computational linguistics* 40, 1 (2014), 121–170.
- [17] Richard Lawrence and Paul Rayner. 2019. *Behavior-Driven Development with Cucumber: Better Collaboration for Better Software*. Addison-Wesley Professional.
- [18] Quentin Limbourg, Jean Vanderdonck, Benjamin Michotte, Laurent Bouillon, and Victor López-Jaquero. 2004. USIXML: A language supporting multi-path development of user interfaces. In *IFIP International Conference on Engineering for Human-Computer Interaction*. Springer, 200–220.
- [19] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn EM van der Werf, and Sjaak Brinkkemper. 2016. The use and effectiveness of user stories in practice. In *International working conference on requirements engineering: Foundation for software quality*. Springer, 205–222.
- [20] Rahul Mohanani, Paul Ralph, Burak Turhan, and Vladimir Mandic. 2021. How Templated Requirements Specifications Inhibit Creativity in Software Engineering. *IEEE Transactions on Software Engineering* (2021).
- [21] David Navarre, Philippe Palanque, Jean-Francois Ladry, and Eric Barboni. 2009. ICOs: A model-based user interface description technique dedicated to interactive systems addressing usability, reliability and scalability. *ACM Transactions on Computer-Human Interaction (TOCHI)* 16, 4 (2009), 1–56.
- [22] Dan North. 2022. What's in a Story? <http://dannorth.net/whats-in-a-story/>
- [23] Oyindamola Olajubu. 2015. A textual domain specific language for requirement modelling. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 1060–1062.
- [24] Gabriel Oliveira and Sabrina Marczak. 2018. On the understanding of BDD scenarios' quality: Preliminary practitioners' opinions. In *International Working Conference on Requirements Engineering: Foundation for Software Quality*. Springer, 290–296.
- [25] Thiago Rocha Silva, Marco Winckler, and Hallvard Trætteberg. 2019. Ensuring the consistency between user requirements and graphical user interfaces: a behavior-based automated approach. In *International Conference on Computational Science and Its Applications*. Springer, 616–632.
- [26] Thiago Rocha Silva, Marco Winckler, and Hallvard Trætteberg. 2019. Ensuring the consistency between user requirements and GUI prototypes: A behavior-based automated approach. In *IFIP Conference on Human-Computer Interaction*. Springer, 644–665.
- [27] Thiago Rocha Silva, Marco Winckler, and Hallvard Trætteberg. 2020. Ensuring the Consistency between User Requirements and Task Models: A Behavior-Based Automated Approach. *Proceedings of the ACM on Human-Computer Interaction* 4, EICS (2020), 1–32.
- [28] Mary Beth Rosson and John M Carroll. 2007. Scenario-based design. In *The Human-Computer Interaction Handbook*. CRC Press, 1067–1086.
- [29] Thiago Rocha Silva and Brian Fitzgerald. 2021. Empirical Findings on BDD Story Parsing to Support Consistency Assurance between Requirements and Artifacts. In *Evaluation and Assessment in Software Engineering*. 266–271.
- [30] Thiago Rocha Silva and Brian Fitzgerald. 2021. Parsing BDD Stories for Automated Verification of Software Artefacts. *Electronic Communications of the EASST* 79 (2021).
- [31] Thiago Rocha Silva, Jean-Luc Hak, and Marco Winckler. 2017. A behavior-based ontology for supporting automated assessment of interactive systems. In *2017 IEEE 11th International Conference on Semantic Computing (ICSC)*. IEEE, 250–257.
- [32] Thiago Rocha Silva, Jean-Luc Hak, and Marco Winckler. 2017. A formal ontology for describing interactive behaviors and supporting automated testing on user interfaces. *International Journal of Semantic Computing* 11, 04 (2017), 513–539.
- [33] Thiago Rocha Silva and Marco Winckler. 2017. A scenario-based approach for checking consistency in user interface design artifacts. In *proceedings of the XVI Brazilian symposium on human factors in computing systems*. 1–10.
- [34] Thiago Rocha Silva, Marco Winckler, and Hallvard Trætteberg. 2019. Extending Behavior-Driven Development for Assessing User Interface Design Artifacts. In *The 31st International Conference on Software Engineering & Knowledge Engineering (SEKE 2019)*.
- [35] Smashtest. 2022. Smashtest. <https://smashtest.io> Last accessed: April 2022.
- [36] Squish. 2022. Automated GUI Testing - Squish GUI Tester. <https://www.froglogic.com/squish/> Last accessed: April 2022.
- [37] Dave Steinberg, Frank Budinsky, Ed Merks, and Marcelo Paternostro. 2008. *EMF: eclipse modeling framework*. Pearson Education.
- [38] Alvaro Veizaga, Mauricio Alferez, Damiano Torre, Mehrdad Sabetzadeh, and Lionel Briand. 2021. On systematically building a controlled natural language for functional requirements. *Empirical Software Engineering* 26, 4 (2021), 1–53.
- [39] Marco Winckler and Philippe Palanque. 2003. StateWebCharts: A formal description technique dedicated to navigation modelling of web applications. In *International Workshop on Design, Specification, and Verification of Interactive Systems*. Springer, 61–76.
- [40] Xtext. 2022. Xtext - Language Engineering Made Easy! <https://www.eclipse.org/Xtext/> Last accessed: April 2022.
- [41] Fiorella Zampetti, Andrea Di Sorbo, Corrado Aaron Visaggio, Gerardo Canfora, and Massimiliano Di Penta. 2020. Demystifying the adoption of behavior-driven development in open source projects. *Information and Software Technology* 123 (2020), 106311.
- [42] Liping Zhao, Waad Alhoshan, Alessio Ferrari, Keletso J. Letsholo, Muideen A. Ajagbe, Erol-Valeriu Chioasca, and Riza T. Batista-Navarro. 2021. Natural Language Processing (NLP) for Requirements Engineering: A Systematic Mapping Study. *Comput. Surveys* 54 (2021). Issue 3.